

Метрики задач работы с изображениями

по строкам - задача, по столбцам - домен

Задачи:	1 Домен - реалистичное изображение, фото	2 Домен - старая или поврежденная фотография, ч/б фото	3 Домен - Предметы искусства - статуи, картины и т.д.	4 Домен - Рисунок - мультипликация и тому подобное	5 Домен - Текстуры	6 Домен - Сканы документов
убери фон						
убери объект						
добавь объект						
реставрируй / восстанови качество						
улучши качество						
раскрась						
измени ракурс						
убери водяной знак						
изменить стиль (рисунок карандашом в акварель, рисунок в фото и так далее)						
совместить части изображения						
Изменить объекты, изменить цвет объектов, дорисовать (голову, лапы, хвост), заменить.						
изменить эмоцию						

1. Индивидуальные метрики

1.1. LPIPS (Learned Perceptual Image Patch Similarity)

LPIPS (Learned Perceptual Image Patch Similarity) — это метрика, которая измеряет **перцептивное расстояние** между двумя изображениями.

Пример использования:

```
import torch
from torchmetrics.image.lpip import LearnedPerceptualImagePatchSimilarity

# ( VGG )
lpips = LearnedPerceptualImagePatchSimilarity(net_type='vgg').cuda()

# [N, 3, H, W] [0, 1] [-1, 1]
# img_original:
# img_generated:

score = lpips(img_generated, img_original)
print(f"LPIPS Distance: {score.item()}")
# < 0.1
# < 0.05
```

1.2. NIQE

1.3. GSB (Good Same Bad) (Human Evaluation)

1.3.1. Общее описание

GSB (Good/Same/Bad) — это метод сравнительной оценки (Side-by-Side, SBS), используемый для измерения относительного качества двух генеративных моделей. В отличие от абсолютных метрик (MOS, FID), GSB не оценивает изображение в вакууме, а определяет, насколько **Модель А (Candidate)** лучше или хуже **Модели В (Baseline)**.

Этот метод является отраслевым стандартом для оценки субъективных качеств: эстетики, соответствия промпту и отсутствия дефектов.

1.3.2. Ключевые состояния

Оценка производится всегда относительно **Модели А** (тестируемой модели):

- **Good (Win):** Результат Модели А явно лучше, чем у Модели В.
- **Same (Tie):** Результаты обеих моделей сопоставимы по качеству (обе отличные или обе плохие), и невозможно выделить явного лидера.
- **Bad (Loss):** Результат Модели А явно хуже, чем у Модели В.

1.3.3. Расчет показателей

Итоговой метрикой обычно является **Net Win Rate** (Чистый коэффициент побед).

$$\text{Net Win Rate} = (\% \text{Good}) - (\% \text{Bad})$$

- Если результат > 0 : Модель А лучше Базовой.
- Если результат < 0 : Модель А хуже Базовой.

Также часто визуализируют распределение всех трех классов (как на диаграмме Nupuruan), чтобы видеть долю "ничьих".

1.3.4. 3. Алгоритм реализации

Процесс оценки требует строгого соблюдения процедуры **Blind Side-by-Side** (слепое сравнение), чтобы исключить предвзятость (bias) оценщика к положению картинки (левая/правая) или названию модели.

1.3.5. Шаг 1: Подготовка данных

Соберите датасет, состоящий из троек:

1. **Prompt:** Текстовое описание.
2. **Image A:** Генерация вашей модели.
3. **Image B:** Генерация модели конкурента (или предыдущей версии вашей модели).

1.3.6. Шаг 2: Анонимизация и перемешивание (Position Shuffling)

Это критический этап. Оценщик (человек или LLM) часто имеет склонность выбирать первый/левый вариант. Для каждой пары необходимо случайно менять местами изображения А и В.

1.4. SSAE (Structured Semantic Alignment Evaluation)

[альтернатива - DREAMBENCH++](#)

1.5. CLIP Score / **directional** CLIP Score

```

import torch
import open_clip
from PIL import Image
import torch.nn.functional as F

class OpenCLIP evaluator:
    def __init__(self, model_name='ViT-L-14', pretrained='laion2b_s32b_b82k', device=None):
        """
        Инициализация OpenCLIP.
        Рекомендуемые модели:
        - Быстрая: ViT-B-32 (pretrained='laion2b_s34b_b79k')
        - Точная: ViT-L-14 (pretrained='laion2b_s32b_b82k')
        - Очень точная: ViT-H-14 (pretrained='laion2b_s32b_b79k')
        """
        self.device = device if device else ("cuda" if torch.cuda.is_available() else "cpu")
        print(f"Loading OpenCLIP model: {model_name} on {self.device}...")

        self.model, _, self.preprocess = open_clip.create_model_and_transforms(
            model_name, pretrained=pretrained
        )
        self.model.to(self.device)
        self.model.eval()
        self.tokenizer = open_clip.get_tokenizer(model_name)

    def _encode_image(self, image):
        """Хелпер для получения нормализованного эмбединга изображения"""
        if isinstance(image, str):
            image = Image.open(image).convert("RGB")

        img_tensor = self.preprocess(image).unsqueeze(0).to(self.device)
        with torch.no_grad():
            features = self.model.encode_image(img_tensor)
            return F.normalize(features, dim=-1)

    def _encode_text(self, text):
        """Хелпер для получения нормализованного эмбединга текста"""
        tokens = self.tokenizer([text]).to(self.device)
        with torch.no_grad():
            features = self.model.encode_text(tokens)
            return F.normalize(features, dim=-1)

    def calculate_clip_score(self, image, prompt):
        """
        Базовая метрика (CLIP Score).
        Используется для задач Генерации (T2I).

        Args:
            image: PIL Image или путь к файлу
            prompt: Текстовый запрос (например, "girl watercolor style")

        Returns:
            float: Косинусное сходство (обычно от 0.15 до 0.35)
        """
        img_emb = self._encode_image(image)
        text_emb = self._encode_text(prompt)

        # Dot product нормализованных векторов = Cosine Similarity
        score = (img_emb @ text_emb.T).item()
        return score

```

```

def calculate_directional_clip_score(self, img_before, img_after, text_before, text_after):
    """
    Продвинутая метрика для задач Редактирования (Image Editing).
    Проверяет, соответствует ли ИЗМЕНЕНИЕ картинки ИЗМЕНЕНИЮ текста.

    Формула: CosSim( (Img2 - Img1), (Text2 - Text1) )

    Пример:
        text_before: "photo of a dog"
        text_after: "photo of a dog with glasses"
        Инструкция: "add glasses"
    """

    # 1. Эмбединги изображений
    i1 = self._encode_image(img_before)
    i2 = self._encode_image(img_after)

    # 2. Эмбединги текстов (описываем состояние ДО и ПОСЛЕ)
    t1 = self._encode_text(text_before)
    t2 = self._encode_text(text_after)

    # 3. Вычисляем дельты (векторы изменений)
    img_direction = F.normalize(i2 - i1, dim=-1)
    text_direction = F.normalize(t2 - t1, dim=-1)

    # 4. Насколько векторы изменений сонаправлены?
    score = (img_direction @ text_direction.T).item()
    return score

# =====
# ПРИМЕР ИСПОЛЬЗОВАНИЯ В ВАШЕМ ПАЙПЛАЙНЕ
# =====

if __name__ == "__main__":
    # 1. Инициализация
    evaluator = OpenCLIP evaluator()

    # --- Сценарий 1: Оценка генерации (Qwen Image) ---
    # Художник попросил: "Cyberpunk city with neon lights"
    prompt = "Cyberpunk city with neon lights"
    gen_image_path = "generated_city.jpg" # Предположим, файл существует

    # Для теста создадим заглушку картинки
    img_gen = Image.new('RGB', (256, 256), color='purple')

    score_gen = evaluator.calculate_clip_score(img_gen, prompt)
    print(f"Generation Score (CLIP): {score_gen:.4f}")

    # --- Сценарий 2: Оценка редактирования (Qwen Image Edit) ---
    # Задача: "Сделай фото черно-белым"
    # Для Directional CLIP нам нужно описать текстом, что было и что стало.

    img_src = Image.new('RGB', (256, 256), color='red') # Было цветное
    img_res = Image.new('RGB', (256, 256), color='gray') # Стало ч/б

    # Описываем семантику изменения
    txt_src = "color photo"
    txt_dst = "black and white photo"

    score_edit = evaluator.calculate_directional_clip_score(
        img_src, img_res,
        txt_src, txt_dst
    )
    print(f"Editing Score (Directional): {score_edit:.4f}")

```

1.6. DINO v2 Score

```

import torch
import torch.nn.functional as F
from transformers import AutoImageProcessor, AutoModel
from PIL import Image

class Dinov2Evaluator:
    def __init__(self, model_name="facebook/dinov2-base", device="cuda"):
        self.device = device
        print(f"Loading DINOv2 model: {model_name}...")
        self.processor = AutoImageProcessor.from_pretrained(model_name)
        self.model = AutoModel.from_pretrained(model_name).to(self.device)
        self.model.eval()

    def get_embedding(self, image):
        """ """
        # (resize, normalize)
        inputs = self.processor(images=image, return_tensors="pt").to(self.device)

        with torch.no_grad():
            outputs = self.model(**inputs)

        # last_hidden_state.
        # DINOv2 [0] CLS ( ),
        # (mean pooling) .
        # DINOv2 CLS .
        last_hidden_states = outputs.last_hidden_state
        cls_token = last_hidden_states[:, 0, :] # [Batch, Hidden_Dim]

        # ! , dot product
        return F.normalize(cls_token, p=2, dim=1)

    def calculate_score(self, image1, image2):
        """ Image """
        emb1 = self.get_embedding(image1)
        emb2 = self.get_embedding(image2)

        # = Cosine Similarity
        score = torch.mm(emb1, emb2.transpose(0, 1))
        return score.item()

# --- ---

#
evaluator = Dinov2Evaluator(device="cuda" if torch.cuda.is_available() else "cpu")

#
img_orig = Image.open("photo.jpg").convert("RGB")
img_edit = Image.open("photo_watercolor.jpg").convert("RGB") # ,

#
similarity = evaluator.calculate_score(img_orig, img_edit)

print(f"DINOv2 Similarity: {similarity:.4f}")

```